

MOBILE PROGRAMMING LAB

Topic: Product Management & Shopping Cart App

Objective

Students will build a complete mobile application using React Native with features such as user registration & login, product display and management, shopping cart, and revenue statistics.

The goal is to practice working with data state management, UI layout, navigation, and product list operations in a cross-platform environment (Android & iOS).

Required Features

1) User Registration

- Input fields: Full Name, Email, Password, Confirm Password.
- Validation:
 - Correct email format.
 - Password and Confirm Password must match.
- Save user data to local database or remote server.

2) User Login

- Login using registered Email & Password.
- Validate with stored data.
- On success → navigate to Product List screen.
- “Remember me” using AsyncStorage.

3) Product List

- Display products with Name, short description, price, and image (if any).
- Use FlatList to render items.

4) Product Management (CRUD)

- **Add Product:** Name, description, price, image (URL or local).

- **View Details:** Tap item to show full details.
- **Update Product:** Edit existing product.
- **Delete Product:** Remove product from list.

5) Product Search

- Search by product name.
- Real-time filtering while typing.

6) Product Sorting

- Sort by price:
 - Ascending
 - Descending

7) Shopping Cart

- Add product to cart from product list or detail.
- Show cart items with quantity & total price.
- Update quantity directly in cart.
- Remove items from cart.
- “Checkout” to simulate order placement.

8) Revenue Statistics

- Record orders after checkout.
- Calculate & display total revenue from completed orders.
- Filter revenue by Day / Month / Year (optional).

UI Requirements

- Modern, intuitive, mobile-friendly UI.
- Navigation between screens with React Navigation.
- Clear feedback messages (Toast/Alert/Modal) for success/error.
- Each feature should be implemented on a separate screen.

Technical Requirements

Students may use one of the following storage options:

- Mock API: **JSON Server**
- Cloud DB: **MongoDB**

Code should be organized into reusable components:

- Forms
- Lists
- Details
- Cart
- Revenue
- Navigation stack